Campus Events

Сервис записи на университетские мероприятия:
модель требований, физическая модель данных, REST API и MCP-сервер

ЛР № 4 — стили интеграции и проектирование API

Группа 13 · индивидуальная работа

Zeng Yaoxin · 2026

ГРАНИЦЫ РАБОТЫ

Что входит в лабораторную

- Методичка: задания **1–5** для групп 10–12, задания **1–4** — для группы 13.
- Обязательный объём — **задания 1–4**; задание 5 (MCP-сервер) выполнено дополнительно.
- Вход — результаты ЛР № 3: **16** историй, **20** вариантов использования, **14** таблиц предметной области.
- Сквозной принцип методички: результат предыдущей работы — вход следующей, поэтому нумерация UC сохранена.

5
заданий закрыто

7
коммитов в main

0
незакрытых issue

РАСПРЕДЕЛЕНИЕ РАБОТ

Кто и что делал

ЗАДАНИЕ	СОДЕРЖАНИЕ	АРТЕФАКТ	ИСПОЛНИТЕЛЬ
1	Модель требований и реестр вариантов использования	`docs/02-requirements.md`	Zeng Yaoxin
2	Логическая и физическая модель БД, DDL, демонстрационные данные	`db/schema.sql, seed.sql, verify.sql`	Zeng Yaoxin
3	Маршруты и конечные точки, коды ошибок, идемпотентность	`docs/06-api.md`	Zeng Yaoxin
4	Спецификация OpenAPI и Mock-сервер	`api/openapi.yaml`	Zeng Yaoxin
5	MCP-сервер для AI-ассистента (дополнительно)	`mcp/server.py`	Zeng Yaoxin
Док.	README, wiki (7 страниц), GitHub Pages, презентация	`docs/, index.html`	Zeng Yaoxin

Работа индивидуальная: разделения ролей внутри команды нет, весь объём заданий 1–5 закрыт одним исполнителем.

ЗАДАНИЕ 1

Требования: что пришлось добавить

- Реестр из трёх колонок: **актор** → **пользовательская история** → **вариант использования**, 20 строк, каждый UC с номером.
- Требования ЛР № 3 описывали интерфейс, а не API. Добавлены **3** сценария: пакетная регистрация, журнал аудита, экспорт отчёта.
- Четыре UC не имели ни одного метода (управление клубами, отчёт, настройки уведомлений, назначение роли) — добавлено **6** операций, итого **29** вместо 23.
- Проверка на разрывы: у каждого UC есть актор и конечная точка, у каждой истории — хотя бы один UC.

АРХИТЕКТУРА

Три слоя и один источник правды

Клиенты

Веб-приложение, мобильное приложение, киоск на входе — обращаются к одному REST-контракту

API · /api/v1

29 операций, 28 схем, bearer JWT, машиночитаемые коды ошибок

PostgreSQL

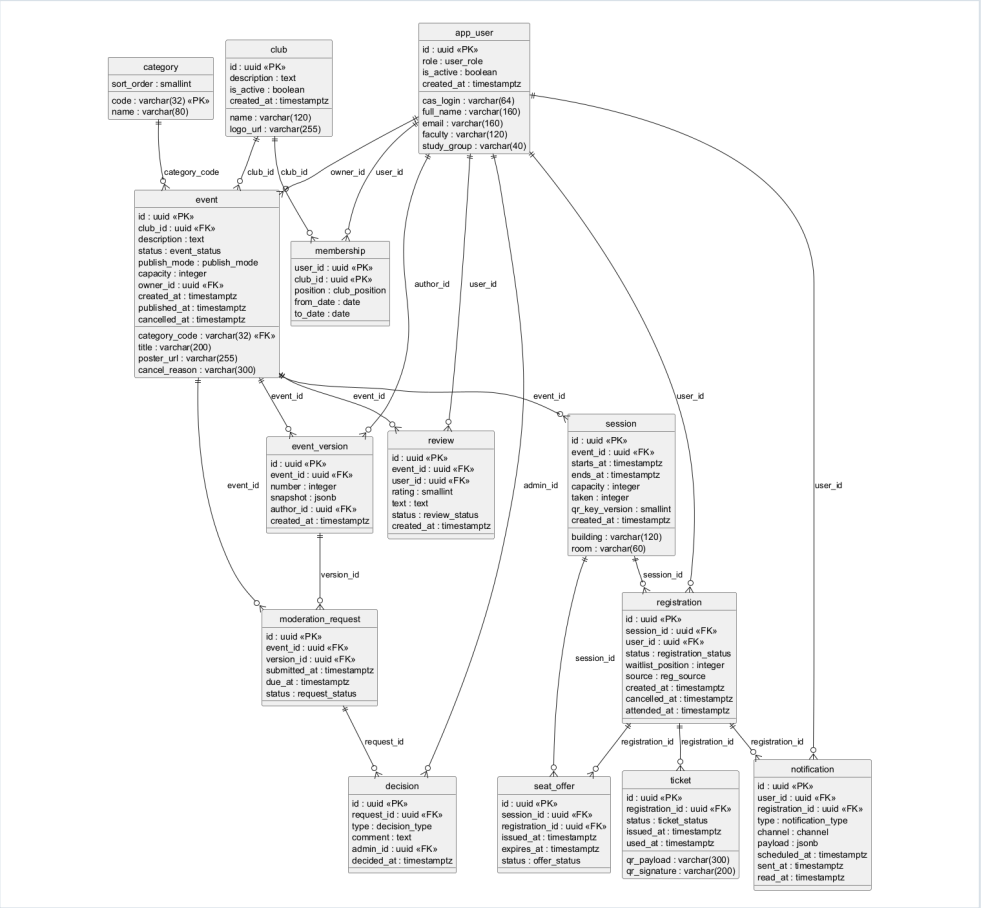
14 таблиц, 13 перечислений, 3 триггера-инварианта; DDL, данные и словарь полей генерируются из одной модели

MCP-сервер читает ту же базу и повторяет бизнес-правила, поэтому AI-ассистент не может обойти ограничения, зашитые в СУБД.

ЗАДАНИЕ 2

Физическая модель данных

- **14** таблиц, **103** поля, **13** перечислений.
- **7** частичных уникальных индексов: активная запись одна на пару «сессия — пользователь».
- **11** CHECK-ограничений вместо проверок в коде приложения.
- Словарь полей с русским смыслом каждого поля — отдельная таблица в документации.



ER-диаграмма физической модели, нотация IE

ЗАДАНИЕ 2

Инварианты живут в базе, а не в коде

- **taken ∈ [0, capacity]** — счётчик занятых мест не может уйти в минус или превысить вместимость.
- **Одна активная запись** на пару «сессия — пользователь»; после отмены повторная запись разрешена.
- **Решения модерации только добавляются** — изменение и удаление запрещены триггером.
- Проверено на работающей СУБД: **15** попыток нарушить ограничения, все отклонены, транзакция откатана.

Скрипт проверки — db/verify.sql — воспроизводим: поднимается одна команда docker compose up.

ЗАДАНИЕ 3 · 4

API: 29 операций и проверяемый контракт

- **25** путей, **29** операций, **28** схем, **5** примеров запросов.
- Коды: 400 / 401 / 403 / 409 / 422 с машинными значениями **NO_SEATS**, **ALREADY_REGISTERED**, **TICKET_USED**.
- Идемпотентность: настройки уведомлений (PUT), повторное сканирование билета.
- Каждый POST, PUT и PATCH имеет описанное тело запроса — требование методички.
- Спецификация проходит официальный валидатор без ошибок.
- Mock-сервер отдаёт 201, 409 и 403 — контракт пригоден для параллельной разработки клиентов.

29
операций

28
схем компонентов

0
ошибок валидатора

ЗАДАНИЕ 5 · ДОПОЛНИТЕЛЬНО

MCP-сервер для AI-ассистента

- **3 инструмента**: создать запись, отменить запись, отметить билет на входе.
- **2 ресурса**: афиша и записи пользователя — без адресов почты и телефонов, как требует спецификация ЛР № 3.
- **1 шаблон промпта**: подсказка по записи сначала читает ресурс, потом отвечает.
- Ошибки возвращаются структурой (**code**, **free_seats**), а не исключением: ассистент сам решает, что предложить — очередь вместо записи.
- **10** сценариев проверки: успех, дубль, заполненная сессия, отмена с передачей места, повторное сканирование.

РАБОТА С РЕПОЗИТОРИЕМ

Как организован процесс

ПРАКТИКА	КАК РЕАЛИЗОВАНО
Ветки и коммиты	Одна ветка **main**, 7 осмысленных коммитов: каждый закрывает одно задание и содержит проверяемый артефакт.
GitHub Projects	Доска с колонками **To do** → **In progress** → **To check** → **Done**, шесть задач по числу заданий и документации.
Issues	Каждая задача оформлена отдельным issue с критериями приёмки; закрывается только после перехода через To check.
Wiki	Семь страниц: главная, требования, схема файлов, дополнительная спецификация, база данных, API, презентация.
GitHub Pages	Статический сайт с той же структурой, что и wiki; страницы API и БД рендерятся из спецификации и модели.

ПРОВЕРЯЕМОСТЬ

Что можно перезапустить за минуту

- **База данных** — docker compose up, затем db/verify.sql: 15 проверок ограничений.
- **Спецификация** — openapi-spec-validator по api/openapi.yaml: 25 путей, 29 операций.
- **MCP-сервер** — mcp/test_mcp.py: 10 сценариев на mock-базе SQLite.
- **Документация** — tools/gen_db.py и tools/gen_site.py пересобирают словарь полей, ER-диаграмму и страницы сайта из одного источника.

Ни одно утверждение в отчёте не проверяется вручную: у каждого числа есть команда, которая его воспроизводит.

ИТОГИ

Что дальше

- Задания **1–4** закрыты полностью, задание **5** — сверх обязательного объёма.
- Ограничение: клиентские приложения (веб и мобильное) остаются отдельными репозиториями и в этой работе не реализованы.
- Следующий шаг — подключение реального PostgreSQL к MCP-серверу вместо mock-базы и публикация клиента на том же контракте.

github.com/1997yyyy/Human-Machine-Interface-Design-LAB4

1997yyyy.github.io/Human-Machine-Interface-Design-LAB4